

المتغيرات وإستخداماتها

5

المتغيرات (variables) هي مفهوم برمجي مهم. إنَّها رموز تدل على البيانات التي نستخدمها في برنامجك، أي تعد حاويات للبيانات التي ستتعامل معها في برنامج. سيغطي هذا الفصل بعض أساسيات المتغيرات، وكيفية استخدامها استخدامًا صحيحًا في برامج بايثون 3.

1. فهم المتغيرات

من الناحية الفنية، يُخصَّص للمتغير مساحة تخزينية في الذاكرة توضع القيمة المرتبطة به فيها. يُستخدم اسم المتغير للإشارة إلى تلك القيمة المُخزَّنة في ذاكرة البرنامج التي هي جزء من ذاكرة الحاسوب. المُتغيِّر أشبه بعنوان تُلصقه على قيمة مُعيَّنة:



لنفترض أنَّ لدينا عددًا صحيحًا يساوي 103204934813، ونريد تخزينه في متغيِّر بدلاً من إعادة كتابة هذا العدد الطويل كل مرة، لذلك سنستخدم شيئًا يُسهِّل تذكُّره، مثل المتغير `my_int`:

```
my_int = 103204934813
```

إذا نظرنا إليه على أنَّه عنوانٌ مرتبط بقيمة، فسيبدو على النحو التالي:



عنوان القيمة هو `my_int` المكتوب عليها، والقيمة هي `103204934813` (نوعها عدد صحيح، سنتطرق إلى أنواع البيانات في الفصل التالي).

التعليمة `my_int = 103204934813` هي تعليمة إسناد (assignment statement)، وتتألف من الأجزاء التالية:

- اسم المتغير (`my_int`)
- معامل الإسناد، المعروف أيضًا باسم "علامة المساواة" (`=`)
- القيمة التي أُسِنِدَت إلى اسم المتغير (`103204934813`)

تشكل هذه الأجزاء الثلاثة معًا التعليمة التي تُسِنِد على المتغير `my_int` القيمة العددية الصحيحة `103204934813`.

بمجرد تعيين قيمة متغير ما، نكون قد هَيَّئْنَا أو أنشأْنَا ذلك المتغير. وبعد ذلك، يمكننا استخدام ذلك المتغير بدلاً من القيمة. في بايثون، لا يلزم التصريح عن المتغيرات قبل استخدامها كما هو الحال في بعض لغات البرمجة الأخرى، إذ يمكنك البدء في استخدام المتغير مباشرةً.

بمجرد إسناد القيمة `103204934813` إلى المتغير `my_int`، يمكننا استخدام `my_int` مكان العدد الصحيح، كما في المثال التالي:

```
print(my_int)
```

والمخرجات هي:

```
103204934813
```

استخدام المتغيرات يسهل علينا إجراء العمليات الحسابية. في المثال التالي، سنستخدم

التعليمة السابقة، `my_int = 1040`، ثم سنطرح من المتغير `my_int` القيمة 813 :

```
print(my_int - 813)
```

وسينتج لنا:

```
103204934000
```

في هذا المثال، يجري بايثون العملية الحسابية، ويطرح 813 من المتغير `my_int`، ويعيد

القيمة 103204934000.

يمكن ضبط المتغيرات وجعلها تساوي ناتج عملية حسابية. دعنا نجمع عددين معًا، ونخزّن

قيمة المجموع في المتغير `x`:

```
x = 76 + 145
```

يشبه المثال أعلاه إحدى المعادلات التي تراها في كتب الجبر. في الجبر، تُستخدم الحروف

والرموز لتمثيل الأعداد والكميات داخل الصيغ والمعادلات، وبشكل مماثل، المتغيرات أسماء

رمزية تمثل قيمة من نوع بيانات معيّن. الفرق في لغة بايثون، أن عليك التأكد دائمًا من وضع

المتغير على الجانب الأيسر من المعادلة.

لنطبع x:

```
print(x)
```

والمخرجات:

221

أعادت بايثون القيمة 221 لأنه أُسند إلى المتغير x مجموع 76 و 145.

يمكن أن تحوي المتغيرات أي نوع من أنواع البيانات، وليس الأعداد الصحيحة فقط:

```

my_string = 'Hello, World!'
myflt = 45.06
mybool = 5 > 9 # True أو False
القيم المنطقية ستعيد إما
my_list = ['item_1', 'item_2', 'item_3', 'item_4']
my_tuple = ('one', 'two', 'three')
my_dict = {'letter': 'g', 'number': 'seven', 'symbol': '&'}
```

إذا طبعت أيًا من المتغيرات المذكورة أعلاه، فستعيد بايثون قيمة المتغير. على سبيل

المثال، في الشيفرة التالية سنطبع متغيرًا يحتوي قائمة:

```

my_list = ['item_1', 'item_2', 'item_3', 'item_4']
print(my_list)
```

وسينتج لنا:

```
['item_1', 'item_2', 'item_3', 'item_4']
```

لقد مررنا القيمة ['item_1' و 'item_2' و 'item_3' و 'item_4'] إلى

المتغير my_list، ثم استخدمنا الدالة print() لطباعة تلك القيمة باستدعاء my_list.

تُخصّص المتغيّرات مساحةً صغيرةً من ذاكرة الحاسوب، وتقبل قيمًا تُوضَع بعد ذلك في تلك المساحة.

2. قواعد تسمية المتغيرات

تتسم تسمية المتغيرات بمرونة عالية، ولكن هناك بعض القواعد التي عليك أخذها في الحسبان:

- يجب أن تكون أسماء المتغيرات من كلمة واحدة فقط (بدون مسافات)
- يجب أن تتكوّن أسماء المتغيرات من الأحرف والأرقام والشرطة السفلية (_) فقط
- لا ينبغي أن تبدأ أسماء المتغيرات برقم

باتباع القواعد المذكورة أعلاه، دعنا نلقي نظرة على بعض الأمثلة:

اسم غير صحيح	اسم صحيح	لماذا غير صالح؟
my-int	my_int	غير مسموح بالشرطات
4int	int4	لا يمكن البدء برقم
\$MY_INT	MY_INT	لا يمكن استخدام أيّ رمز غير الشرطة السفلية
another int	another_int	لا ينبغي أن يتكون الاسم من أكثر من كلمة واحدة

من الأمور التي يجب أخذها في الحسبان عند تسمية المتغيرات، هو أنَّها حساسة لحالة الأحرف، وهذا يعني أنَّ `my_int` و `MY_INT` و `My_Int` و `mY_iNt` كلها مختلفة. ينبغي أن تتجنب

استخدام أسماء متغيرات متماثلة لضمان ألا يحدث خلط عندك أو عند المتعاونين معك، سواء الحاليين والمستقبليين.

أخيرًا، هذه بعض الملاحظات حول أسلوب التسمية. من المتعارف عليه عند تسمية المتغيرات أن تبدأ اسم المتغير بحرف صغير، وأن تستخدم الشرطة السفلية عند فصل الكلمات. البدء بحرف كبير مقبول أيضًا، وقد يفضل بعض الأشخاص استعمال تنسيق سنام الجمل (camelCase، الخلط بين الأحرف الكبيرة والصغيرة) عند كتابة المتغيرات، ولكن هذه الخيارات أقل شهرة.

تنسيق غير شائع	تنسيق شائع	لماذا غير متعارف عليه
myInt	my_int	أسلوب سنام الجمل (camelCase) غير شائع
Int4	int4	الحرف الأول كبير
myFirstString	my_first_string	أسلوب سنام الجمل (camelCase) غير شائع

الخيار الأهم الذي عليك التمسك به هو الاتساق. إذا بدأت العمل في مشروع يستخدم تنسيق سنام الجمل في تسمية المتغيرات، فمن الأفضل الاستمرار في استخدام ذلك التنسيق. يمكنك مراجعة توثيق [تنسيق الشيفرات البرمجية في بايثون](#) في موسوعة حسوب للمزيد من التفاصيل.

3. تغيير قيم المتغيرات

كما تشير إلى ذلك كلمة "متغير"، يمكن تغيير قيم المتغيرات في بايثون بسهولة. هذا يعني أنه يمكنك تعيين قيمة مختلفة إلى متغير أُسندت له قيمة مسبقًا بسهولة بالغة. القدرة على إعادة إسناد القيم مفيدة للغاية، إذ قد تحتاج خلال أطوار برنامجك إلى قبول قيم ينشئها المستخدم وتحويلها على متغير. سهولة إعادة إسناد المتغيرات مفيدة أيضًا في البرامج الكبيرة التي قد تحتاج خلالها إلى تغيير القيم باستمرار.

سنُسنِد إلى المتغير `x` أولًا عددًا صحيحًا، ثم نعيد إسناد سلسلة نصية إليه:

```
# تعيين x إلى قيمة عددية
x = 76
print(x)

# إعادة تعيين x إلى سلسلة نصية
x = "Sammy"
print(x)
```

وسينتج لنا:

```
76
Sammy
```

يوضح المثال أعلاه أنه يمكننا أولًا إسناد قيمة عددية إلى المتغير `x`، ثم إعادة إسناد قيمة

نصية إليه.

إذا أعدنا كتابة البرنامج بالشكل التالي:


```
x = 76
x = "Sammy"
print(x)
```

لن نتلقى سوى القيمة المسندة الثانية في المخرجات، لأنَّ تلك القيمة هي الأحدث:

```
Sammy
```

قد تكون إعادة إسناد القيم إلى المتغيرات مفيدة في بعض الحالات، لكن عليك أن تبقي عينك على مقروئية الشيفرة، وأن تحرص على جعل البرنامج واضحًا قدر الإمكان.

4. الإسناد المتعدد (Multiple Assignment)

في بايثون، يمكنك إسناد قيمة واحدة إلى عدة متغيرات في الوقت نفسه. يتيح لك هذا تهيئة عدَّة متغيرات دفعةً واحدةً، والتي يمكنك إعادة إسنادها لاحقًا، أو من خلال مدخلات المستخدم.

يمكنك من خلال الإسنادات المتعددة إسناد قيمة واحدة إلى عدَّة متغيرات (مثل المتغير

x و y و z) في سطر واحد:

```
x = y = z = 0
print(x)
print(y)
print(z)
```

النتاج سيكون:

```
0
0
0
```

عرفنا في هذا المثال ثلاثة متغيرات (x و y و z)، وأسندنا إليها القيمة 0.

تسمح لك بايثون أيضًا بإسناد عدّة قيم لعدّة متغيّرات ضمن السطر نفسه. هذه القيم يمكن

أن تكون من أنواع بيانات مختلفة:

```
j, k, l = "shark", 2.05, 15
print(j)
print(k)
print(l)
```

وهذه هي المخرجات:

```
shark
2.05
15
```

في المثال أعلاه، أُسندت السلسلة النصية "shark" إلى المتغير j ، والعدد العشري 2.05

إلى المتغير k ، والعدد الصحيح 15 إلى المتغير l .

إسناد عدة متغيرات بعدة قيم في سطر واحد يمكن أن يختصر الشيفرة ويقلل من عدد

أسطرها، ولكن تأكد من أنّ ذلك ليس على حساب المقروئية.

5. المتغيرات العامة والمحلية

عند استخدام المتغيرات داخل البرنامج، من المهم أن تضع نطاق (scope) المتغير في

حساباتك. يشير نطاق المتغير إلى المواضع التي يمكن الوصول منها إلى المتغيّر داخل الشيفرة،

لأنّه لا يمكن الوصول إلى جميع المتغيرات من جميع أجزاء البرنامج، فبعض المتغيرات عامة

(global)، وبعضها محلي (local). مبدئيًا، تُعرّف المتغيرات العامة خارج الدوال؛ أمّا المتغيرات

المحلية، فتعرّف داخل الدوال.

المثال التالي يعطي فكرة عن المتغيرات العامة والمحلية:

```
# إنشاء متغير عام، خارج الدالة
glb_var = "global"

def var_function():
    # إنشاء متغير محلي داخل دالة
    lcl_var = "local"
    print(lcl_var)

# استدعاء دالة لطباعة المتغير المحلي
var_function()

# طباعة متغير عام خارج دالة
print(glb_var)
```

المخرجات الناتجة:

```
local
global
```

يُسند البرنامج أعلاه سلسلة نصية إلى المتغير العمومي `glb_var` خارج الدالة، ثم يعرف الدالة `var_function()`. وسيُنشئ داخل تلك الدالة متغيرًا محليًا باسم `lcl_var`، ثم يطبعه قيمته. ينتهي البرنامج باستدعاء الدالة `var_function()`، وطباعة قيمة المتغير `glb_var`.

لما كان `glb_var` متغيرًا عامًا، فيمكننا الوصول إليه داخل الدالة `var_function()`.

المثال التالي يبيّن ذلك:

```
glb_var = "global"

def var_function():
    lcl_var = "local"
```

```
print(lcl_var)
print(glb_var) # طباعة glb_var داخل الدالة

var_function()
print(glb_var)
```

المخرجات:

```
local
global
global
```

لقد طبعنا المتغير العام `glb_var` مرتين، إذ طُبع داخل الدالة وخارجها.
ماذا لو حاولنا استدعاء المتغير المحلي خارج الدالة؟

```
glb_var = "global"

def var_function():
    lcl_var = "local"
    print(lcl_var)

print(lcl_var)
```

لا يمكننا استخدام متغير محلي خارج الدالة التي صُرح عنه فيها. إذا حاولنا القيام بذلك،

فسيتُطلق الخطأ `NameError`.

```
NameError: name 'lcl_var' is not defined
```

دعنا ننظر إلى مثال آخر، حيث سنستخدم الاسم نفسه لمتغير عام وآخر محلي:

```

num1 = 5 # متغير عام

def my_function():
    num1 = 10 # استخدام نفس اسم المتغير num1
    num2 = 7 # تعيين متغير محلي

    print(num1) # طباعة المتغير المحلي num1
    print(num2) # طباعة المتغير المحلي num2

# استدعاء my_function()
my_function()

# طباعة المتغير العام num1
print(num1)

```

الناتج:

```

10
7
5

```

نظرًا لأنَّ المتغير المحلي num1 صُرِّحَ عنه محليًّا داخل إحدى الدوال، فسنرى أنَّ num1 يساوي القيمة المحلية 10 عند استدعاء الدالة. عندما نطبع القيمة العامة للمتغير num1 بعد استدعاء الدالة my_function()، سنرى أنَّ المتغير العام num1 لا يزال مساويًا للقيمة 5.

من الممكن الوصول إلى المتغيرات العامة واستعمالها داخل دالة باستخدام الكلمة المفتاحية global:

```
def new_shark():
    # جعل المتغير عاما
    global shark
    shark = "Sammy"

# استدعاء الدالة
new_shark()

# طباعة المتغير العام
print(shark)
```

رغم أنَّ المتغير المحلي shark عُيِّن داخل الدالة new_shark()، إلا أنَّه يمكن الوصول إليه من خارج الدالة بسبب الكلمة المفتاحية global المستخدمة قبل اسم المتغير داخل الدالة. بسبب استخدام global، فلن يُطْلَق أيُّ خطأ عندما نستدعي print(shark) خارج الدالة. رغم أنَّه يمكنك استعمال متغير عام داخل دالة، إلا أنَّ ذلك يُعدُّ من العادات غير المستحبة في البرمجة، لأنَّها قد تؤثر على مقروئية الشيفرة عدا عن السماح لجزء من الشيفرة بتعديل قيمة متغير قد يستعمله جزء آخر.

هناك شيء آخر يجب تذكره، وهو أنَّك إذا أشرت إلى متغير داخل دالة، دون إسناد قيمة له، فسيُعدُّ هذا المتغير عامًا ضمنيًا. لجعل متغيرٍ محليًا، يجب عليك إسناد قيمة له داخل متن الدالة. عند التعامل مع المتغيرات، يكون لك الخيار بين استخدام المتغيرات العامة أو المحلية. يُفضَّل في العادة استخدام المتغيرات المحلية، ولكن إن وجدت نفسك تستخدم نفس المتغير في عدة دوال، فقد ترغب في جعله عامًا. أمَّا إن كنت تحتاج المتغير داخل دالة أو صنف واحد فقط، فقد يكون الأولى استخدام متغير محلي.

6. خلاصة الفصل

لقد مررنا في هذا الفصل على بعض حالات الاستخدام الشائعة للمتغيرات في بايثون 3. المتغيرات هي لبنة مهمة في البرمجة، إذ تُمثِّل حاضنةً لمختلف أنواع البيانات في بايثون والتي سنسلط عليها الضوء في الفصل التالي.